RP_v2

1. Title

Task-Free Online LoRA with Drift Detection and Inference-Time Routing for Continual Instruction Tuning

2. Abstract

Large language models (LLMs) deployed in real-world applications face non-stationary instruction streams where user intents, instruction formats, and domains shift over time. In such settings, task boundaries are typically unknown (task-free), while storing large replay buffers is often constrained by privacy and cost. Naïve sequential fine-tuning leads to catastrophic forgetting, degrading performance on earlier instructions. Inspired by task-free online continual learning with low-rank adaptation, we propose a task-free online LoRA framework for Continual Instruction Tuning (CIT). Our method combines: (i) **curriculum-aware drift detection** based on a small public anchor instruction set and sequential change-point statistics with automatic calibration, (ii) a **unified LoRA bank + routing loop** that isolates adaptation across detected phases by freezing past LoRA branches and instantiating new ones while training a lightweight router for task-free inference-time selection, and (iii) an **anti-overlap regularization** that encourages functional specialization of LoRA branches in activation space to prevent redundant experts and unstable routing. We will evaluate on the CITB benchmark streams (InstrDialog / InstrDialog++) and report average performance, forgetting, anytime performance, drift detection quality, and routing quality under strict memory budgets.

3. Background and Motivation

3.1 Real-world problem: non-stationary instruction streams

Modern LLM applications continuously receive new instruction data (e.g., new product features, policy changes, evolving user preferences). This creates a **non-stationary** data stream where the underlying distribution changes over time, and explicit task boundaries are rarely available.

3.2 Why naive fine-tuning fails: catastrophic forgetting

Continual learning literature shows that sequentially learning new data can overwrite parameters important for earlier data, leading to catastrophic forgetting. Regularization-based methods (e.g., Elastic Weight Consolidation, EWC) penalize changing important parameters to retain past knowledge.

3.3 Why parameter-efficient adaptation (LoRA) is a good fit

Full fine-tuning is expensive and hard to maintain across many updates. LoRA freezes base weights and learns low-rank updates, dramatically reducing trainable parameters while keeping inference efficient.

4. Related Work

LoRA / PEFT. LoRA learns low-rank updates to Transformer weights to enable efficient adaptation without full fine-tuning.

Continual Instruction Tuning (CIT). CITB establishes benchmark streams (InstrDialog / InstrDialog++) and protocols for studying continual instruction tuning; it finds many CL methods do not effectively leverage natural language instructions and that sequential fine-tuning can be competitive.

LoRA-based continual learning and routing. Existing LoRA-CL methods often allocate a new LoRA per task and may introduce gating to reduce forgetting (e.g., GainLoRA), but typically rely on task boundaries or task identities during training/evaluation.

MoE-style routing between LoRA experts and identity paths has also been explored to suppress forgetting (e.g., SLIM).

Task-free online continual learning. Online-LoRA proposes a task-free online CL framework (in vision) using training dynamics to recognize distribution shifts and a novel online regularization strategy to consolidate important parameters.

5. Research Gap and Key Questions

5.1 Gap

Current LLM continual instruction tuning methods often assume known task boundaries, or they do not provide an explicit mechanism to (i) detect distribution shifts reliably, (ii) allocate new adaptation capacity when needed, and (iii) perform task-free inference-time selection among multiple specialized adapters under memory constraints. In addition, as a LoRA bank grows, **overlap between branches** can cause unstable routing and mixed behaviors, yet this is rarely measured or controlled explicitly.

5.2 Key questions

1. **Drift detection:** Can we detect meaningful distribution shifts in a task-free instruction stream using a small public anchor set, with low false-alarm rate and low detection delay?
2. **Capacity allocation:** When drift is detected, how should we expand LoRA capacity while preventing interference with earlier behaviors?
3. **Task-free inference:** Without task IDs, can a lightweight router reliably select the correct LoRA branch (or mixture) at inference time?
4. **Specialization vs overlap:** As the bank grows, how can we encourage **functional specialization** of LoRA branches to reduce redundancy and prevent unstable routing?

6. Proposed Method

6.1 Overview

We use a single frozen base model f_θ(Llama-3.1-8B-Instruct) and maintain a growing LoRA bank $\{\phi^{(1)}, \ldots, \phi^{(m)}\}$. A drift detector monitors a fixed public anchor set and triggers creation of a new LoRA branch when a change-point is detected. At inference time, task identifiers are unavailable, so we train a lightweight router $g_\psi$to perform hard routing (top-1 selection) over LoRA branches for each input. we add LoRA-only importance regularization to further stabilize past capabilities.

6.2 LoRA parameterization (core formula)

For a Transformer linear layer $W \in \mathbb{R}^{d\times k}$, LoRA parameterizes an update ΔWas low-rank:
$W_{\text{new}} = W + \Delta W, \quad \Delta W = BA,$
where $A \in \mathbb{R}^{r\times k}$, $B \in \mathbb{R}^{d\times r}$, and $r \ll \min(d, k)$. We freeze W and train only (A,B).

6.3 Drift detection via anchor monitoring + curriculum-aware sequential change statistics
Anchor set (two-tier curriculum).

We construct a public anchor set of size M (default 128; ablation over M). We split it into (i) a stable core subset (domain-agnostic, format-stable instructions) and (ii) a sensitive probe subset (format/domain-sensitive instructions) to improve robustness and reduce false alarms under noisy short-term fluctuations. The anchor set is used only for monitoring, never for training updates.

Monitoring statistics. Every K training steps (default 100), we compute the anchor

average token-level NLL for both subsets, denoted as $\bar{s}core and\ \bar{s}probe$.

Change-point rule (CUSUM with automatic calibration). We use CUSUM-style sequential monitoring on $\bar{s}core\ and\ \bar{s}probe$, and trigger drift only when the probe indicates sustained degradation and the core shows consistent deviation beyond a calibrated tolerance. We calibrate thresholds using an initial window, and optionally apply meta-recommendation to select CUSUM hyperparameters (e.g., threshold and slack) across multiple synthetic/held-out streams to minimize detection delay under a controlled false-alarm budget.
Trigger action. When drift is detected, we freeze the current branch and instantiate a new branch $\phi^{(m+1)}$initialized to zero.

6.4 Unified LoRA bank management + inference-time routing (closed-loop)

We maintain a bank of LoRA branches$\{\phi^{(1)}, \ldots, \phi^{(m)}\}$. For stability, completed branches $\phi^{(1..m-1)}$ are frozen after their phase ends; only the active branch $\phi^{(m)}$is updated online on incoming batches.
Closed-loop pipeline. Our system forms a loop: Detect → Allocate → Train → Route → Feedback. (i) Drift detection (Sec. 6.3) decides when to freeze the current branch and allocate a new branch; (ii) the active branch is trained online; (iii) a lightweight router performs task-free selection at inference time; (iv) router training uses signals produced by the evolving LoRA bank, with stabilization to avoid label oscillation.
Router model. At inference time, task IDs are unknown. We learn a router $g_\psi$that maps input features z(x) to mixture weights over LoRA branches: $w(x) = \mathrm{softmax}(g_\psi(z(x)))$. We implement z(x) as mean-pooled last-layer hidden states of the frozen base model.
Self-supervised router training (stabilized pseudo-label). For each training sample (x,y), we compute per-branch losses $\ell(f_{\theta,\phi^{(j)}}(x), y)$. We define pseudo-label $j^*(x,y) = \arg\min_j \ell(\cdot)$ and train the router with cross-entropy. To reduce instability as the bank evolves, we optionally (a) use a delayed/EMA copy of branches as the pseudo-label oracle, and/or (b) filter low-margin examples where

the best and second-best branches have similar losses. This requires no task labels.
Connection to prior routing work. Gating/routing has been shown effective for LoRA-based continual learning (e.g., GainLoRA) and MoE-style routing with identity bypass (SLIM).

6.5 Anti-overlap regularization for LoRA specialization (activation-space diversity)

A practical risk in a growing LoRA bank is content overlap: different branches may learn redundant behaviors, causing unstable routing and mixed outputs. To encourage specialization, we add a lightweight diversity regularizer in activation space rather than weight space.
Activation diversity loss. For the same input batch, we run multiple branches and extract hidden representations $h^{(j)}(x)$(e.g., mean-pooled last-layer states). We penalize high cosine similarity between branches to reduce overlap:

$\mathcal{L}div = \sum j \neq k \cos(h^{(j)}(x), h^{(k)}(x)).$

The final objective becomes $\mathcal{L} = \mathcal{L}NLL + \beta\mathcal{L}div.$
Why activation-space. Recent analyses suggest that weight-space orthogonality does not reliably translate into activation-space separation; thus we directly regularize activations to encourage functional diversity.

7. Experimental Plan

7.1Benchmark and setup

Base model: Llama-3.1-8B-Instruct.
Benchmark: CITB streams InstrDialog and InstrDialog++.
Setting: task-free, streaming training, memory-light.
Anchor: M=128 (ablation 64/128/256).

Monitoring interval: K=100.
Routing: hard routing (top-1).

Memory Budgets (fixed)： We evaluate replay sizes in $\{0, 10, 50\}$. When replay is enabled, we maintain a FIFO buffer and mix replay at a fixed ratio (e.g., 1 replay batch per 5 stream batches).

7.2 Baselines

1. Sequential LoRA (single branch, no drift, no router)
2. Replay LoRA (buffer 10/50)
3. Periodic Multi-LoRA (open a new LoRA every fixed interval; total branches matched to our method; always use latest)
4. Bank + no router (drift + bank, but always use latest at inference)
5. Router-only (fixed #branches, router without drift detection)
6. Ours (drift + bank + hard router; importance regularization)

7.3.1 Metrics

- **Average performance** over evaluation checkpoints (CITB standard).
- **Forgetting**: drop from best historical performance on earlier segments.
- **Anytime performance**: performance as a function of stream time (online curve).
- **Drift detection quality**: false alarm rate, miss rate, detection delay (using annotated synthetic shifts or proxy labels).
- **Routing quality**: agreement with oracle j^*, entropy of routing weights, and per-branch utilization.
- **Overlap / specialization (new):** activation similarity across branches on a fixed probe set; correlation with routing stability.

7.3.2 Figures

Figure 1: anytime performance curve (score vs stream step)
Figure 2: mean forgetting bar plot (plus optional segment heatmap)
Figure 3: performance vs memory budget (0/10/50) and vs #branches
Figure 4: drift trigger points over time with anchor statistic s_t
Figure 5: router-oracle agreement and branch utilization histogram

Figure 6: (new, required for full paper): pipeline figure illustrating the online loop (Detect → Allocate → Train → Route → Feedback) and clearly marking frozen vs updated components.

Table 1: main results (avg performance, mean forgetting, final-step performance)

7.4 Ablations (what to remove)

1. **w/o drift detector** (fixed schedule; keep bank+router)
2. **w/o router** (always use latest; keep drift+bank)
3. **w/o LoRA bank** (single LoRA; keep drift logic off)
4. **drift statistic variant**: anchor-NLL vs training-loss plateau
5. **w/o anti-overlap loss** (remove activation diversity)
6. **budget sweep**: max #LoRAs, rank r, anchor size M, monitor interval K

7.5 Timeline

- Week 1–2: implement drift detection (core/probe anchors), baseline training loop, logging.
- Week 3: implement unified bank + router training (stabilized pseudo-label).
- Week 4: implement overlap metrics + activation diversity loss; run ablations.
- Week 5: full benchmark runs under memory budgets; produce figures.
- Week 6: write-up + final comparisons.

Week 1 — Reproducible training/evaluation pipeline + core baselines

Tasks

- Finalize a single config-driven training script (seeded, reproducible) with standardized logging and checkpointing.
- Implement evaluation that reports **(i)** average performance over seen segments, **(ii)** forgetting, and **(iii)** anytime performance curve.
- Run **Sequential LoRA** (single branch) and **Replay LoRA** (buffer 10/50).

Deliverables

- results_baseline_seq.csv, results_baseline_replay.csv
- 1–2 plots: anytime performance (score vs stream time) and forgetting summary
- Clean run artifacts: config.yaml, train.log, metrics.jsonl, checkpoints/

Expected outcomes

- Sequential LoRA shows measurable forgetting over time.
- Replay improves stability but has a clear memory/quality tradeoff, establishing a strong reference point.

Week 2 — Curriculum-aware drift monitoring + drift-triggered LoRA bank (no router yet)

Tasks

- Construct a public anchor set and split into **core/probe** subsets.
- Implement anchor monitoring every K steps (core/probe NLL curves).
- Implement a CUSUM-style drift statistic with calibration and logging of drift events.
- Connect drift triggers to **LoRA bank actions**: freeze current branch → spawn new branch.
- Run **Periodic Multi-LoRA (latest-only)** baseline for a fair capacity control.

Deliverables

- anchor_monitor.jsonl, drift_events.jsonl, branch_registry.json

- Diagnostic figure: drift statistic over time with trigger markers
- Results table comparing: Sequential vs Periodic-latest vs Drift+Bank (latest-only)

Expected outcomes

- Probe anchors are more sensitive than core anchors (more responsive to shifts).
- Drift-triggered bank opens branches less arbitrarily than fixed schedules and reduces forgetting relative to Sequential.

Week 3 — Unified bank + inference-time hard routing (task-free selection)

Tasks

- Implement router features z(x) (e.g., pooled last-layer hidden states from the frozen base model).
- Train a lightweight router using pseudo-labels $j^* = \arg\min_j \ell_j$ with one simple stabilization (e.g., margin filtering).
- Add **hard routing** (top-1) at inference time for efficiency.
- Run **Router-only (fixed branches)** baseline to isolate routing benefits.

Deliverables

- router_metrics.csv (router accuracy vs oracle, entropy/utilization stats)
- Results table and plot including: Periodic-latest, Router-only, Drift+Bank+Router (Ours)

Expected outcomes

- Router-only provides a measurable gain over periodic-latest (or reveals its limits).
- Ours improves anytime performance and reduces forgetting compared to baselines under matched branch budgets.

Week 4 — Overlap diagnostics + activation diversity regularization + minimal ablations

Tasks

- Implement an **overlap metric** (activation cosine similarity across branches on a fixed probe set).
- Add **activation diversity loss** $\mathcal{L}_{div}$ and run a small \beta sweep.
- Run only the most critical ablations:
  - w/o drift detector (fixed schedule)
  - w/o router (always use latest)
  - w/o anti-overlap loss (\beta=0)

Deliverables

- Overlap heatmap or summary statistic + router utilization plots
- Ablation table reporting: score, forgetting, #branches, overlap metric

Expected outcomes

- Anti-overlap loss reduces branch redundancy (lower overlap) and improves routing stability (clearer utilization / lower entropy), ideally with performance gains.

Week 5 — Full benchmark runs under memory budgets + paper-grade figures

Tasks

- Run full benchmark suites under memory budgets (e.g., replay 0/10/50 where applicable).
- Ensure fair matching across methods (max #branches, LoRA rank r, anchor size M, monitor interval K).
- Produce final figures: anytime performance, forgetting, drift diagnostics, router behavior, overlap.

Deliverables

- results_budget_sweep.csv + reproducible figure scripts
- Final figure set (PDF/PNG): main curves + ablation/budget comparisons

Expected outcomes

- A clear performance profile showing when our method beats replay and when replay is competitive, with diagnostic evidence explaining why.

Week 6 — Write-up + final comparisons (paper-ready packaging)

Tasks

- Write Method + Experiments + Discussion (including failure cases and limitations).
- Add a pipeline figure illustrating **Detect** → **Allocate** → **Train** → **Route** → **Feedback**, clearly marking frozen vs trainable components.
- Finalize comparisons and ensure all numbers are script-generated.

Deliverables

- Submission-ready draft (PDF) with tables/figures integrated
- Reproducibility checklist (configs, seeds, scripts, artifacts)

Expected outcomes

- A complete, coherent narrative: (1) drift detection is meaningful, (2) bank allocation prevents interference, (3) routing enables task-free inference, and (4) overlap regularization improves specialization/stability.

8. Expected Contributions

- A **task-free online LoRA** framework for continual instruction tuning with **curriculum-aware drift detection** and **inference-time routing**.
- A **closed-loop bank-routing system** (detect/allocate/train/route/feedback) that improves stability and interpretability in task-free settings.
- An **anti-overlap regularization** (activation-space diversity) and corresponding overlap metrics to analyze and reduce redundant branches.

- Systematic evaluation of drift detection and routing behaviors, beyond end accuracy, under strict memory budgets on CITB.

9. Risks and Mitigations

- **Risk 1: noisy drift triggers / false alarms.** Mitigation: core/probe anchor split; calibrated thresholds; meta-recommendation for hyperparameters.
- **Risk 2: router instability due to evolving bank.** Mitigation: EMA oracle and/or margin filtering for pseudo-label training.
- **Risk 3: branch overlap causing mixed behavior.** Mitigation: overlap metrics + activation diversity loss.
- **Risk 4: compute cost due to multiple branches.** Mitigation: hard routing default; freeze old branches; ablate bank size policies.

10. References

- Hu, E. J., et al. **LoRA: Low-Rank Adaptation of Large Language Models.** arXiv:2106.09685.
- Wei, X., Li, G., & Marculescu, R. **Online-LoRA: Task-free Online Continual Learning via Low Rank Adaptation.** arXiv:2411.05663.
- Zhang, Z., Fang, M., Chen, L., & Namazi-Rad, M.-R. **CITB: A Benchmark for Continual Instruction Tuning.** arXiv:2310.14510 (Findings of EMNLP 2023).
- Kirkpatrick, J., et al. **Overcoming catastrophic forgetting in neural networks (EWC).** arXiv:1612.00796 / PNAS 2017.
- Liang, Y.-S., & Li, W.-J. **GainLoRA: Gated Integration of Low-Rank Adaptation for Continual Learning of Language Models.** arXiv:2505.15424.
- Han, J., et al. **SLIM: Let LLM Learn More and Forget Less with Soft LoRA and Identity Mixture.** NAACL 2025.
- Page, E. S. **CUSUM** (sequential change-point monitoring; original 1954). (Use as concept reference; modern overview acceptable.)

- Soviany, P., Ionescu, R. T., Rota, P., & Sebe, N. **Curriculum Learning: A Survey.** arXiv:2101.10382.
- Lopes, J. F., et al. **Online Meta-Recommendation of CUSUM Hyperparameters** (2025). (PMC article.)
- Kim, H. **Geometric Regularization in Mixture-of-Experts: The Disconnect Between Weights and Activations.** arXiv:2601.00457 (2026).
- Mu, S., & Lin, S. **A Comprehensive Survey of Mixture-of-Experts: Algorithms, Theory, and Applications.** arXiv:2503.07137 (2025).